Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

Alt programlar

- Alt programlar, programların temel yapı taşlarıdır.
- Programlama dillerinde tasarımın en kritik kavramlarından biri olarak kabul edilir.
- Alt programlar sayesinde:
  - Kod tekrarını önleme.
  - Bellek ve zaman tasarrufu.
  - Programın mantıksal yapısının okunabilirliği artar
- İlgili kavramlar: parametre geçiş yöntemleri, yerel değişkenler, aşırı yükleme, genel alt programlar, yan etkiler.
- Soyutlama
- İki temel soyutlama vardır:
  - İşlem soyutlaması: Alt programlar aracılığıyla sağlanır.
  - Veri soyutlaması: 1980'lerden sonra önem kazanmıştır.Tarihsel örnek: Babbage'in Analytical Engine (1840'lar) → tekrar kullanılabilir komut kartları.
- Modern alt programlar, belirli bir hesaplamayı soyutlayarak program içinde çağrılabilir hale getirir.

Alt programlar

- Kodun tekrar kullanımı → bellek ve geliştirme süresinde tasarruf sağlar.
- Soyutlama: Alt programın iç detayları gizlenir, çağıran program sadece sonucu kullanır.
- Okunabilirliği artırır: Programın mantıksal yapısı öne çıkar, düşük seviyeli detaylar saklanır.
- Nesne yönelimli dillerde yöntemler, alt programların sınıf ve nesne ile ilişkili formudur.

- Her alt programın tek giriş noktası vardır.
- Alt program çalışırken çağıran program askıya alınır
  - → tek seferde bir alt program çalışır.
  - Alt program tamamlandığında kontrol geri döner.
- Çoğu alt program isimlidir; bazıları anonim olabilir (C#, Python, Lua, Ruby).
- Özel durumlar: Coroutine ve eşzamanlı programlama (concurrency) da alt programlar aynı anda çalıştırılabilir.

Alt programlar

- Başlık genelde alt programın türünü belirtir (fonksiyon, prosedür).
- İsim ve parametre listesi içerir.

```python
def adder(x, y):
    return x + y
```

- Gövde (Body): Alt programın işlemlerini tanımlar.
  - C, JavaScript → {} kullanılır.
  - Python → girintileme ile belirlenir.
  - Ruby → end ile kapanır

- Formal parametre: Alt program başlığında tanımlanan değişkenler (dummy değişkenler).
- Actual parametre: Alt program çağrılırken verilen değerler.
- Parametre bağlama yöntemleri:
  - Pozisyonel: Sıralamaya göre bağlanır (en yaygın).
  - Anahtar kelime: İsim üzerinden bağlanır → sıralama önemli değil.
    - Örnek Python keyword parametre:

```python
sumer(length=my_length, list=my_array, sum=my_sum)
```

Alt programlar

- **Varsayılan parametre:** Çağrıda verilmezse başlıkta belirtilen değer kullanılır.
- **Değişken sayıda parametre:**
  - C → printf
  - C# → params
  - Ruby → esnek parametre listesi, hash ve array ile
  - Lua → ... kullanılır

```python
def compute_pay(income, exemptions=1, tax_rate=0.15):
    return income - exemptions*tax_rate
```

Fonksiyonlar ve Prosedürler

- Fonksiyon: Bir değer döndürür.
- Prosedür: Sadece işlem yapar, değer döndürmez.
- Nesne yönelimli dillerde yöntemler (method): Parametre ve yerel değişkenler alt program ile aynı şekilde çalışır.
  - Ana fark sınıf ve nesne ile ilişkili olmasıdır.
  - Yan etkiler: Fonksiyonlar dış değişkenleri değiştirebilir bu da güvenilirlik sorunları.

- Alt program isimleri, başka alt programlara parametre olarak verilebilir.
- Fonksiyonel programlama → yan etkisiz ve güvenli hesaplamalar.
- Ruby ve Python, hem pozisyonel hem keyword parametrelerle değişken sayıda parametreyi destekler.

Ruby

```ruby
list = [2,4,6,8]
def tester(p1,p2,p3,*p4)
  # işlemler
end
tester('first', mon:72, tue:68, wed:59, *list)
```

Fonksiyonlar ve Prosedürler

- Prosedürler, çağrıldıkları program birimine iki yöntemle sonuç döndürebilir:
  - Görünür Değişkenler (Non-formal Parameters): Prosedür ve çağıran program biriminde ortak görünürlüğe sahip olan (global veya non-local) değişkenler üzerinden. Prosedür bu değişkenlerin değerini değiştirebilir.
  - Biçimsel Parametreler (Formal Parameters): Veri aktarımına izin veren parametreler üzerinden. Bu parametrelerin değeri çağıran birime geri taşınabilir.
- Saf Fonksiyon (Pure Function): Matematiksel fonksiyon modeline sadık kalır. Hiçbir yan etkisi yoktur. Ne parametrelerini ne de fonksiyon dışında tanımlanmış değişkenleri değiştirir. Tek etkisi döndürdüğü değerdir.

- Pratikte: Çoğu programlama dilinde fonksiyonlar yan etkilere sahiptir.

- Kullanıcı Tanımlı Operatörler: Fonksiyonlar, yeni kullanıcı tanımlı operatörler olarak düşünülebilir.

- Örnek: Bir dilde üs alma operatörü yoksa, bir fonksiyon yazılabilir.

- C++: float power(float base, float exp) tanımlanıp result = 3.4 * power(10.0, x) çağrılabilir. (C++ kütüphanesinde pow vardır).
- Perl: result = 3.4 * 10.0 ** x (Üs alma yerleşik bir işlemdir).

Alt programlar ve Fonksiyonlar

- Alt programlar karmaşık yapılardır. Tasarımlarında ele alınması gereken temel sorular:
  - Yerel değişkenler statik mi yoksa stack-dynamic mi ayrılır?
  - Alt program tanımları başka alt program tanımları içinde yer alabilir mi? (Nested subprograms)
  - Hangi parametre geçiş yöntemi/yöntemleri kullanılır?
  - Aktüel parametrelerin türleri, biçimsel parametrelerin türleriyle kontrol edilir mi? (Type checking)
- Alt programlar parametre olarak geçilebilir mi? Geçilebiliyorsa ve nested subprogramlar varsa, geçirilen alt programın referans ortamı (referencing environment) nedir?
- Alt programlar aşırı yüklenebilir mi? (Overloading)
- Alt programlar genel (generic) olabilir mi?
- Nested subprogramlar destekleniyorsa, closure'lar desteklenir mi?
- Closure: Bir nested subprogram ve onun referans ortamının birleşimidir. Alt programın programın herhangi bir yerinden çağrılabilmesini sağlar.

Yerel Referans ve Değişkenler

- Yerel değişkenler, alt program içinde tanımlanır. Kapsamları (scope) genellikle tanımlandıkları alt programın gövdesidir.
- Stack-Dynamic: Alt program çağrıldığında belleğe bağlanır, sonlandığında bellekten ayrılır.
  - Avantajlar: Özyineleme (recursion) için gereklidir; bellek paylaşımı sağlar.
  - Dezavantajlar: Her çağrıda ayırma/başlatma/sonlandırma maliyeti; dolaylı erişim (indirect access) gerektirir; history-sensitive olamaz (çağrılar arasında değer saklayamaz).
- Statik (Static): Derleme zamanında ayrılır, program sonlanana kadar varlığını sürdürür.
  - Avantajlar: Daha hızlı erişim (doğrudan); history-sensitive olabilir.
  - Dezavantajlar: Özyinelemeyi desteklemez; bellek paylaşımı yapamaz.

```c
int adder(int list[], int listlen) {
    static int sum = 0; // Statik
    int count;          // Stack-dynamic
    for (count = 0; count < listlen; count++)
        sum += list[count];
    return sum;
}
```

Alt Programlar

- Nested subprograms: Alt programların içinde başka alt program tanımlanmasıdır. Algol 60 ile başlamıştır. Statik kapsamlamayı (static scoping) destekler.
- Destekleyenler: Algol 68, Pascal, Ada, JavaScript, Python, Ruby, Lua.
- Desteklemeyenler: C ve türevleri (C++, Java, C# metod içinde metod tanımına izin vermez).

Alt programlar

- **Python'da Yerel Değişkenler:**
  - Metotlarda sadece global bildirimi vardır. global olarak bildirilmeyen bir değişkene atama yapılırsa, o değişken yerel (stack-dynamic) kabul edilir.
  - Metot dışındaki bir değişken, global bildirimi olmadan okunabilir ama atanamaz.
- **Lua'da Yerel Değişkenler:**
  - local sum gibi bildirimle yerel değişken tanımlanır. Yerel değişkenlere erişim, global değişkenlere erişimden daha hızlıdır.
  - Bildirimi olmayan tüm değişkenler globaldir.

Parametre Geçiş Yöntemleri

- Üç Temel Semantik Model:
  - In mode: Biçimsel parametre, aktüel parametreden veri alır.
  - Out mode: Biçimsel parametre, aktüel parametreye veri gönderir.
  - Inout mode: Biçimsel parametre, aktüel parametreye veri alır ve gönderir.
- Kavramsal Modeller:
  - Değer Kopyalama (Copy): Aktüel değer çağırana, çağrılana veya her iki yöne kopyalanır.
  - Erişim Yolu (Access Path): Basit bir işaretçi (pointer) veya referans iletilir.

Parametre Geçiş Yöntemleri

- Pass-by-Value: in mode'u uygular. Aktüel parametrenin değeri, biçimsel parametreyi başlatmak için kullanılır.
  - Avantaj: Skalerler için hızlıdır.
  - Dezavantaj: Büyük verilerde kopyalama maliyeti ve ek bellek gerektirir.
- Pass-by-Result: out mode'u uygular. Alt program sonlanmadan hemen önce biçimsel parametrenin değeri aktüel parametreye geri kopyalanır.
  - Problem: Aynı aktüel parametrenin iki kez kullanılması durumunda geri kopyalama sırası belirsizliği

```
void Fixer(out int x, out int y) { x = 17; y = 35; }
// ...
f.Fixer(out a, out a); // a'nın değeri implementasyona bağlı (17 veya 35)
```

- Pass-by-Value-Result: inout mode'u uygular. Pass-by-value ve pass-by-result'un birleşimidir. (Copy-in, copy-out)

Parametre Geçiş Yöntemleri

- Pass-by-Reference: inout mode'u uygular. Aktüel parametrenin adresi (erişim yolu) iletilir. Biçimsel parametre, aktüel parametre ile aynı bellek hücresini paylaşır.
  - Avantaj: Zaman ve bellek açısından verimlidir (kopyalama yok).
  - Dezavantajlar:
    - Dolaylı adresleme nedeniyle erişim daha yavaştır.
    - Sadece veri alınacaksa (in mode), yanlışlıkla değişiklik yapılabilir.
    - Alias (Takma Ad) Problemleri: Aynı değişkenin iki farklı parametre ile geçilmesi, aynı dizi elemanının geçilmesi, dizi ve dizi elemanının birlikte geçilmesi veya global değişkenlerle parametrelerin çakışması durumunda alias oluşur.
- Örnek (C++): void fun(int &first, int &second) ve fun(total, total) çağrısında first ve second alias'tır.
- Pass-by-Value-Result kullanımı alias sorununu ortadan kaldırır, ancak farklı problemler getirebilir.

Parametre Geçiş Yöntemleri

- C: Pass-by-value. Pass-by-reference için işaretçiler (pointers) kullanılır.
- C++: Pass-by-value. Referans tipi (reference type) ile pass-by-reference sağlanır. const anahtar sözcüğü ile write-protection (in mode) uygulanabilir.

```
void fun(const int &p1, int p2, int &p3)
```

(p1: sabit ref, p2: değer, p3: ref)

- Java: Her şey pass-by-value. Nesneler referans değişkenleriyle erişildiği için, nesne parametreleri etkili bir şekilde pass-by-reference gibi davranır (referansın kopyası gider, ancak nesne değiştirilebilir).
- C#: Varsayılan pass-by-value. ref ile pass-by-reference, out ile out-mode (pass-by-reference) sağlanır.
- PHP: & ile pass-by-reference belirtilir.
- Perl: Tüm aktüel parametreler @_ adlı özel bir diziye konur. Dizi elemanları aktüel parametrelerin alias'larıdır. Bu dizide yapılan değişiklikler aktüel parametreleri etkiler.
- Python / Ruby: Pass-by-assignment. Tüm veriler nesnedir, her değişken bir referanstır. Aktüel parametrenin referans değeri, biçimsel parametreye atanır. Bu etkili bir şekilde pass-by-reference'dır. Ancak değiştirilemez (immutable) nesnelerde (örneğin skalerler) yapılan değişiklikler, yeni nesne oluşturulduğu için aktüel parametreyi etkilemez.

Parametre Geçiş Yöntemleri - C

```c
#include <stdio.h>

// Pass-by-value: x'in kopyası gelir, orijinal değişmez
void passByValue(int x) {
    x = 100;
}

// İşaretçi ile pass-by-reference taklidi: adres kopyalanır, ancak adres üzerinden
orijinal değişken değiştirilebilir
void passByPointer(int *x) {
    *x = 100;  // Orijinal değişkenin değerini değiştirir
}

int main() {
    int a = 10;

    passByValue(a);
    printf("passByValue sonrasi a = %d\n", a);  // Çıktı: 10 (değişmedi)

    passByPointer(&a);
    printf("passByPointer sonrasi a = %d\n", a); // Çıktı: 100 (değişti)

    return 0;
}
```

```cpp
// Pass-by-value: kopya oluşur
void passByValue(int x) {
    x = 100;
}

// Pass-by-reference: orijinal değişken üzerinde çalışır
void passByReference(int &x) {
    x = 100;
}

// const reference: in-mode, büyük verilerde kopyalama maliyetini önler, ancak değ
işikliğe izin vermez
void printAndProtect(const int &x) {
    // x = 100; // HATA! const referans değiştirilemez
    cout << "Değer: " << x << endl;
}

int main() {
    int a = 10;

    passByValue(a);
    cout << "passByValue sonrasi a = " << a << endl;  // 10

    passByReference(a);
    cout << "passByReference sonrasi a = " << a << endl; // 100

    printAndProtect(a);  // Çıktı: Değer: 100
```

Parametre Geçiş Yöntemleri - Java

```java
// İlkel tip: pass-by-value (kopya gelir)
static void changePrimitive(int x) {
    x = 100;
}

// Nesne referansı: referansın değeri kopyalanır
static void changeObject(Person p) {
    p.name = "Ali";  // Nesnenin içeriği değiştirilebilir
}

// Referansın kendisini değiştirmeye çalışmak
static void reassignReference(Person p) {
    p = new Person("Veli");  // Bu sadece yerel kopyayı etkiler
}
```

```java
    public static void main(String[] args) {
        int a = 10;
        changePrimitive(a);
        System.out.println("changePrimitive sonrasi a = " + a);  // 10 (değişmedi)

        Person ali = new Person("Ahmet");
        changeObject(ali);
        System.out.println("changeObject sonrasi ali.name = " + ali.name);  // Ali
(değişti)

        reassignReference(ali);
        System.out.println("reassignReference sonrasi ali.name = " + ali.name);
// Ali (değişmedi, referans hala aynı nesneyi gösteriyor)
    }
```

Parametre Geçiş Yöntemleri – CSharp

```csharp
using System;

class Program
{
    // Pass-by-value (varsayılan)
    static void PassByValue(int x)
    {
        x = 100;
    }

    // Pass-by-reference (ref ile)
    static void PassByReference(ref int x)
    {
        x = 100;  // Orijinal değişkeni değiştirir
    }

    // Out-mode (out ile) - giriş değeri gerekmez, sadece çıkış
    static void OutMode(out int x, out int y)
    {
        x = 17;   // Out parametreleri kesinlikle atanmalı
        y = 35;
    }
```

```csharp
    static void Main()
    {
        int a = 10;
        PassByValue(a);
        Console.WriteLine($"PassByValue sonrası a = {a}");  // 10

        PassByReference(ref a);
        Console.WriteLine($"PassByReference sonrası a = {a}");  // 100

        // out kullanımı
        int result1, result2;
        OutMode(out result1, out result2);
        Console.WriteLine($"OutMode sonrası: result1 = {result1}, result2 = {resul
t2}");  // 17, 35

        // Metinde belirtilen collision problemi
        int collision = 21;
        void Fixer(out int x, out int y) { x = 17; y = 35; }
        Fixer(out collision, out collision);
        Console.WriteLine($"Collision sonrası collision = {collision}");  // Imple
mentasyona bağlı (17 veya 35)
    }
}
```

Parametre Geçiş Yöntemleri - Php

```php
<?php
// Pass-by-value (varsayılan)
function passByValue($x) {
    $x = 100;
}

// Pass-by-reference (& ile)
function passByReference(&$x) {
    $x = 100;
}

$a = 10;
passByValue($a);
echo "passByValue sonrası a = $a\n";  // 10

passByReference($a);
echo "passByReference sonrası a = $a\n";  // 100

// Metinde belirtilen: ya aktüel ya da formal parametrede & kullanılabilir
function flexible($x, &$y) {
    $x = 50;
    $y = 60;
}
```

Parametre Geçiş Yöntemleri - Python

```python
def modify_immutable(x):
    x = 100  # Yeni nesne oluşturur, orijinal referans etkilenmez

def modify_mutable(lst):
    lst.append(4)  # Mutable nesnenin içeriği değiştirilebilir

def reassign_mutable(lst):
    lst = [10, 20, 30]  # Yerel referansı yeniden atar, orijinal etkilenmez

# Immutable nesne (int)
a = 10
modify_immutable(a)
print(f"modify_immutable sonrası a = {a}")  # 10 (değişmedi)

# Mutable nesne (list)
b = [1, 2, 3]
modify_mutable(b)
print(f"modify_mutable sonrası b = {b}")  # [1, 2, 3, 4] (değişti)

reassign_mutable(b)
print(f"reassign_mutable sonrası b = {b}")  # [1, 2, 3, 4] (değişmedi, hala aynı l
iste)
```

Parametre Geçiş Yöntemleri

Dil	Varsayılan	Pass-by-Reference	Out-Mode	Özel Notlar
C	Pass-by-value	İşaretçilerle taklit	Yok	
C++	Pass-by-value	Referanslar (`&`) ile	Yok	`const` ile in-mode
Java	Pass-by-value	Yok (referansların değeri kopyalanır)	Yok	Nesne içeriği değiştirilebilir
C#	Pass-by-value	`ref` ile	`out` ile	
PHP	Pass-by-value	`&` ile	Yok	Aktüel veya formalde `&`
Perl	Pass-by-value	`@_` dizisi alias sağlar	Yok	`@_` elemanları doğrudan alias
Python	Pass-by-assignment	Mutable nesnelerde etkili	Yok	Immutable nesnelerde değişiklik olmaz
Ruby	Pass-by-assignment	Mutable nesnelerde etkili	Yok	Python ile benzer

Özet

- **Alt Programlar:** Prosedürler (değer döndürmez, yeni ifadeler tanımlar) ve fonksiyonlar (değer döndürür, yeni operatörler tanımlar) olarak ikiye ayrılır.
- **Tasarım Konuları:** Yerel değişkenlerin tahsisi, nested yapılar, parametre geçiş yöntemleri, tür kontrolü, aşırı yükleme, generic yapılar ve closure'lar alt program tasarımının temel sorularıdır.
- **Parametre Geçişi:** Semantik olarak *in, out, inout* modları; uygulama olarak *pass-by-value, pass-by-result, pass-by-value-result, pass-by-reference* modları vardır.
  - **Pass-by-value:** Hızlı ama kopyalama maliyetli.
  - **Pass-by-reference:** Verimli ama alias sorunlarına yol açabilir.
- **Diller Arası Farklılıklar:** C/C++ referans ve işaretçiler, Java referansların kopyası, C# ref/out, Perl @_ dizisi, Python/Ruby pass-by-assignment gibi farklı mekanizmalar sunar.
- **Sonuç:** Alt programlar, dilin ifade gücünü artıran, modülerliği ve yeniden kullanılabilirliği sağlayan temel yapılardır. Tasarımları, dilin performans, güvenlik, esneklik ve okunabilirlik hedeflerine göre şekillenir.